

DBCLS BioHackathon 2026 report: Template for the very long title

Miguel Mazumder ¹ and Last Author ²

BioHackathon series:

[DBCLS BioHackathon 2026](#)

Matsuyama, Japan, 2026

[data-engine-evaluation](#)

Submitted: 18 Sep 2026

License:

Authors retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC-BY](#)).

Published by [BioHackrXiv.org](#)

1 Glycan and Life Systems Integration Center (GaLSIC), Soka University, Tokyo, Japan **2** ELIXIR Europe 

Introduction

As part of the DBCLS BioHackathon 2026, we here report our efforts to develop tools and workflows to extract target data from a triple store with complex schema structures. (italicized portion especially should be checked for correctness) The `_GaLSIC_INSERT DESCRIPTION OF GLYCOSMOS PORTAL. INSERT LARGE SCALE GOAL FOR CREATED A HUMAN SPECIFIC GLYCOBIOLOGY DATA SOURCE. AND WHY WE NEE`

LLMs can navigate complexity in a highly organized and complex triple system quickly and accurately. Recognizing the advantages of this quality can improve efficiency of data transfer and data management between users and institutions. To that end, we developed a workflow for agent-assisted data extraction and a companion SPARQL endpoint tool with extended capabilities to facilitate cooperative work between subject matter experts to improve the supervision and validation of that agentic pipeline.

INSERT STUFF ABOUT HOW THIS LEADS INTO ENGINE DEVELOPMENT AND DATA WAREHOUSE MANAGEMENT HOUSEKEEPING

Abstract

The Human Glycome Atlas Project aims to integrate human glycoscience data into a linked RDF resource connecting glycans and glycoconjugates with their biological context. Constructing such a resource from existing knowledge bases requires reproducible identification and extraction of human-relevant data, preservation of semantic behavior when the resulting RDF is served through different graph engines, and appropriate control over access to protected data. During DBCLS BioHackathon 2026, we developed an agent-assisted, human-validated workflow for exploring complex RDF schemas and generating SPARQL CONSTRUCT extraction queries. We further developed a cross-engine conformance framework for evaluating RDF serving behavior and advanced the AAIL authorization architecture used to mediate access to TOHSA data. Together, these activities connect source-data discovery, reproducible extraction, semantic serving, and governed access into a common workflow for constructing and operating the TOHSA knowledge base.

Agentic Assisted Data Extraction

The workflow implemented at BioHackathon is an exchange of supervised task execution by the agent, and validation by the subject matter experts. The workflow (depicted below) consists of agent exploration, human validation, agent creation of extraction queries, human evaluation of those queries, and finally agent-facilitated querying of the endpoint with CONSTRUCT queries resulting in data files written in .ttl format. These files are meant to be compatible with mathematic evaluation via Exploratory Data Analysis (EDA) and/or semantic evaluation

via human-facilitated querying. Developing the criteria for those evaluation steps was outside the scope of this BioHackathon, but it is planned as a future step for this project. All steps are intended to be iterable within and across themselves to allow for dynamic and documented changes resulting from discoveries about the data, changes in desired target or scope, or adjustment

INSERT `extraction_workflow.png` IMAGE

Multi-Step Schema Exploration

To provide understanding about the structure of the linked data, a three-step analysis is applied to each dataset. The agent determines which files in the context repository feed into which named graph, what predicate points to taxonomic information, and what corresponding object marks a resource as human. This is done while consulting the available config files, and ultimately confirming the veracity of all observations with live queries. The exploration queries return the full human IRI count, picking one richly connected, real instance and following every predicate out from it to determine range or relevant triples. This path exploration identifies what is safe to include in an extraction scope, what is a shared resource across graphs and potentially needs its own table, and whether or not a predicate carries a fan-out risk.

For each step, the agent creates an `.md` file that describes observations about the data structure. This analysis is not static, and if future steps reveal something previously unknown or poorly categorized, the agent returns to the `.md` document and amends it before the task is declared complete and the full `.md` file shared with the user. The details included in those observations are described below:

Step One, extraction criteria. Defines which predicate and value combination marks a resource as whatever the intended extraction target, in this case as human. This step runs queries with the intention of proving every claim with a live SPARQL query, and states plainly where a criterion rests in a dataset's documented scope. These queries are saved as separate files in the appropriate directory, and are directly hyperlink-referenced in the observations summary.

Step Two, entry counts. Runs count queries for target criterion from step one against the full live dataset, no `LIMIT`, and records the returned count with the query used to obtain it. Where more than one independent signal exists for the same criterion, this step check them against each other and investigates any disagreement with additional queries to the live endpoint rather than picking one arbitrarily.

Step Three, predicate discovery. Selects one richly linked, already confirmed instance and enumerates every predicate on every resource directly reachable from it, unrestricted. This is where fan out hubs, a predicate that looks safe on one instance but explodes at scale because the target resource is itself cited by thousands of other records, opaque identifiers with no further content, and mislabeled predicates get caught, before they become a silent gap or an unbounded query in a production pipeline.

Below is a summarized example of the conclusions from an agent-assisted exploration of multiple datasets:

Dataset	Criterion	Count	Key finding
Disease	Two levels, DOID membership (concept) plus gene or glycoprotein glycan:has_taxon (molecular evidence)	4115 of 4372	The invented pipeline:hasPhenotype predicate was found and replaced with the real four hop bridge

Dataset	Criterion	Count	Key finding
Pathways	biopax3:organism, direct, confirmed two independent ways	2870 of 23486	Three pipeline bugs found in a deep cross check, a self referencing related pathway bug, a missed nested sub pathway reaction gap, and an uncollected output variable
Genes	glycan:has_taxon directly on glycan:Glycogene	10276	A ggdb (GlycoGene- DataBase) sub analysis was built separately on that named graph's own rich reaction content
Glycoproteins	glycan:has_taxon directly on glycan: Glycoprotein	16711	A numeric pattern gene target's own triples span six graphs, not two; a related_graphs sub analysis covered four further graphs (gpdb, HPA, lipidmaps_gene, protein_egf)
Lectins	glycan:has_taxon directly on sugarbind:Lectin	318 of 6298	CarboGrove, reached through one rdfs:seeAlso hop, expands into 1.3 million triples, a confirmed fan out hub, excluded
Glycans	Two hop glycan:is _from_source/gly can:has_taxon	8042 of 265401	A directory named external holds none of the resource type its name implies; the extraction package's own inference dependent query proved unnecessary; two pipeline bugs found and fixed

Dataset	Criterion	Count	Key finding
Glycolipids	None exists, confirmed by exhaustive live checking	6046 (full population, no species filter possible) of 6280 store wide	No species predicate anywhere in the dataset; the scope decision to cover the full population was made without a response from the person directing the work, and is flagged for confirmation

CONSTRUCT Query Drafting

Once the subject matter confirms that the agent has an accurate representation of the data structure and how the target data fits into that structure, the agent begins to draft construct queries. Those queries are crafted by.....

INSERT INFORMATION FROM AGENT

The user evaluates the query for a) appropriate scope b) time-out risk and c) extraction coverage completeness. Despite being more computationally intensive than SELECT files, CONSTRUCT queries do not require an additional step of triple reconstruction across tables, and ensure fidelity between target triples and the extracted triples. All drafted CONSTRUCT queries are documented before being run against the endpoint, allowing for a subject matter expert to run the query with LIMIT conditions and evaluate the output.

FUTURE CAPABILITIES The documentation created by this workflow is prose heavy and could be simplified into a more structured format that is still human readable, but easier for other agents to engage with. Standard RDF-config files in .yaml format are already commonly used to document RDF schema, and could serve as an appropriate addition to the prose. The simplified and structured .yaml files could facilitate new ways to extend the workflow beyond extraction as we continue development.

The extraction workflow therefore produces both the target RDF and a documented record of how the extraction scope was identified and validated. Future conversion of parts of this documentation into structured configuration files could make those decisions easier for other agents and tools to reuse. Extraction, however, is only the first step in incorporating these data into TOHSA. Once an extracted RDF dataset has been validated, the intended scientific relationships need to be represented consistently, the data need to be served without unintended changes in query results, and access-controlled data need to be exposed only through the appropriate authorized context. These requirements led to additional work on materialization, RDF engine evaluation, and the AAll authorization and serving architecture.

RDF Engine Evaluation and Semantic Serving

The extraction workflow identifies which triples should be included in a human-focused dataset. Once those triples are incorporated into TOHSA, a different set of questions needs to be addressed. The same scientific data may be loaded into different RDF database engines, but those engines do not always treat graph construction, RDF terms, or SPARQL operations in exactly the same way. We therefore evaluated how TOHSA can separate the scientific meaning of the data from the behavior of the engine used to serve it.

Materialization Before Serving

TOHSA combines RDF from multiple sources and may also create additional statements through ontology, mapping, or propagation rules. One option is to calculate those additional statements when a user submits a query. This makes the returned result dependent on the inference capabilities and configuration of the engine answering the query.

We instead explored materializing the required derived statements before the data are served. In this approach, the original statements and the derived statements intended to be part of TOHSA are written into a canonical release before that release is loaded into a database engine. Scientific inference can then be disabled during normal query serving.

This separates two problems that were initially being treated together. The first is whether the RDF release contains the scientific statements that TOHSA intends to provide. The second is whether an RDF engine correctly serves that release through the supported SPARQL query interface. Materialization addresses the first problem, but it does not guarantee the second.

A synthetic release was created to test this approach before applying it to the larger extracted GlyCosmos dataset. The release could be rebuilt deterministically from the same inputs, and the derived statements were stored directly in the resulting RDF. This also allowed representation decisions to be tested independently from engine behavior. For example, integer lexical forms could be normalized during release construction while preserving their RDF datatype identity. Decimal and floating-point normalization were not included because the meaning of lexical precision for measurement values still needs to be determined.

RDF Engine Conformance Evaluation

After materialization was separated from query-time inference, the next step was to determine whether different RDF engines could serve the same release with the behavior required by TOHSA. A conformance test suite was created for this purpose. The same synthetic release, test queries, and expected results were used for Virtuoso, QLever, and RDF4J NativeStore.

The tests covered areas where engine differences can change the result of a query, including default and named graph construction, RDF term identity, and SPARQL aggregation. The intention was not to determine which engine was generally better. The goal was to determine whether each engine satisfied the specific behavior required by the current TOHSA service profile.

Virtuoso showed a problem when the same triple occurred in more than one graph selected into the default dataset. The overlapping graphs could produce duplicate matches where RDF merge behavior should result in one triple. Dataset controls could correct which graphs were treated as default or named graphs, but did not correct this overlapping-graph behavior.

QLever handled the tested overlapping graph case correctly, but did not preserve some of the integer datatype distinctions present in the release. It also returned a different datatype for some aggregate results. Both Virtuoso and QLever changed the lexical form of the decimal value used in the test fixture.

RDF4J NativeStore was then tested as a third implementation. This was useful for determining whether failures seen in Virtuoso and QLever were common to all RDF engines or were specific to a particular implementation. RDF4J preserved the tested integer datatype and decimal lexical distinctions. It reproduced the overlapping graph behavior seen with Virtuoso and showed a separate counting difference for an aggregate over a successful result containing no bound variable.

No engine passed every required case in the current direct-serving profile. The failure patterns were also different between engines. This is important because it means that correcting results after they are returned from the engine is not a general solution. Adding `DISTINCT`, removing duplicate rows, or changing returned RDF datatypes could also change valid query results.

For this reason, the release and the query service are treated as separate responsibilities. The release defines the scientific RDF and any approved representation rules. The service profile defines the query behavior that TOHSA expects from an engine serving that release. An engine can only be used for that profile after its behavior has been tested against those requirements.

The synthetic test suite is intended to remain in use after real TOHSA data are available because each test isolates a specific RDF or SPARQL behavior. The next engine evaluation will use a canonical GlyCosmos-derived human dataset from the extraction work and will add queries based on the structure and scientific content of that dataset.

AAll Authorization and Serving Boundary

Correct engine behavior does not determine whether a user is allowed to access a particular dataset. Authorization is handled separately by the TOHSA Authentication and Authorization Infrastructure (AAll), which sits between the requesting application and the RDF engine.

Earlier versions of the query path authorized a request and then transformed the SPARQL query before sending it to an engine. During the BioHackathon, this boundary was made more explicit. The authorization result is now retained as a structured query plan that includes the parsed query and the default and named graph sets that were authorized.

The engine adapter uses this authorized plan to create the query that will actually be sent to Virtuoso or QLever. Before execution, a verifier checks that the dataset and query being dispatched still match the authorized plan. This prevents a later transformation or engine-specific implementation detail from silently expanding the data available to the query.

Tests against running Virtuoso and QLever containers showed that unauthorized graph scope could be rejected before the query was executed. These tests evaluate authorization enforcement and are separate from the engine conformance tests described above. An engine may correctly receive only the graphs a user is authorized to access and still fail one of the semantic or query behavior requirements in the conformance suite.

The next step is to bind authorization to an identified release and to verify that the selected database target is actually serving that release. Prototype work demonstrated that a restricted reader can obtain release identity information from an isolated Virtuoso target without giving the reader unrestricted database or container-management access.

A separate unresolved question is who has the authority to publish the grants that connect a user to a particular protected data context. Existing agreements and configuration files may provide evidence for that decision, but the software should not automatically interpret them as live authorization. The governance process for publishing those grants remains to be defined.

Protected Data and Workspace Isolation

Some TOHSA data may have access restrictions and cannot be exposed through the same unrestricted serving environment as public data. We therefore also explored whether protected RDF should be separated physically rather than relying only on query-time filtering inside a shared RDF database.

One prototype represented an authorized data context as a Workspace with an immutable WorkspaceRelease and a separate serving target. The prototype tested deterministic release identity, materialization over a defined set of inputs, separation between public and protected targets, rejection of release mismatches, and reconstruction of a serving target from the release artifacts.

This prototype used a strong isolation model in which the data needed for a protected context could be placed into its own serving target. It does not establish the final TOHSA production storage model. In particular, it is still undecided whether a protected context should contain

its own copy of public data, whether public and protected data should be combined through another controlled mechanism, or whether another physical arrangement should be used.

The main result from this work is that protected data should not depend only on every query being correctly filtered inside a shared RDF store. Stronger physical separation can reduce the number of places where a mistake could expose protected data. The exact method used to combine public and protected knowledge still needs to be decided.

The Workspace model also separates the contents of a scientific data context from the permissions given to a user. A WorkspaceRelease identifies a specific set of data. An Access Grant describes what a particular user is allowed to do with that context. This allows multiple users to access the same release under different permissions without making a different scientific release for each user.

Together, these steps extend the extraction workflow into the serving side of TOHSA. The extraction process determines which source data are in scope. Materialization determines which derived statements are included before serving. Engine conformance testing checks whether an RDF engine can serve that release with the required behavior. AAll and the Workspace work then address which data context a user is allowed to query and how protected data can be isolated from unrestricted access.

Ontological Data Investigation Nexus (ODIN)

INSERT LABELED SCREENSHOT HERE

...

Acknowledgements

...

References

Author contributions

Miguel Mazumder: Conceptualization, Methodology, Software, Validation, Data curation, Writing – original draft; Last Author: Conceptualization, Writing – review & editing.



Appendices

If you want the Appendix (-ces) to show up after the references, wrap them in after the header, like done in this Markdown file. Look at the [source](#) to see the exact structure.